RAG Pipeline Step 1: Indexing

1. Definition

Indexing is the process of loading, transforming, and storing data from various sources into a specialized database. This process makes the data searchable for the later "Retrieval" step. It's the foundational, offline process that prepares your knowledge base.

2. Core Process

The most common indexing pipeline, particularly for vector databases, involves four key stages:

1. **Load:** Ingesting raw data from its source. This can be from PDFs, text files, APIs, web pages, or a SQL database.
2. **Split:** Breaking down large documents into smaller, meaningful chunks. This is critical because:
   - It respects the context-window limits of embedding models.
   - It allows for more precise retrieval (finding a specific paragraph is better than finding a 50-page document).
3. **Embed:** Using an embedding model (e.g., from OpenAI, Hugging Face, or Google) to convert each text chunk into a high-dimensional numerical vector. This vector represents the semantic meaning of the text.
4. **Store:** Saving these vectors, along with their corresponding text and metadata, into a database.

3. Tools & Frameworks

Stage	Tool/Concept	Reason for Use (in LangChain)
Load	**Document Loaders** (`WebBaseLoader` , `PyPDFLoader` , `DirectoryLoader`)	LangChain provides a vast collection of loaders to handle almost any data source, abstracting away the complexity of file I/O and data extraction.
Split	**Text Splitters** (`RecursiveCharacterTextSplitter` , `TokenTextSplitter`)	These tools intelligently divide text. `RecursiveCharacterTextSplitter` is often the best choice as it tries to split on logical separators (like paragraphs, sentences, and words) to keep related text together.
Embed	**Embedding Models** (`OpenAIEmbeddings` , `HuggingFaceEmbeddings`)	This is the engine that creates the semantic vectors. LangChain provides a standard interface to use dozens of different embedding providers, making it easy to swap models.
Store	**Vector Stores** (`Chroma` , `FAISS` , `Pinecone` , `Weaviate`)	These databases are specialized for storing and querying high-dimensional vectors. LangChain acts as an abstraction layer, allowing you to use any of

		these backends with a consistent API (e.g., `vectorstore.add_documents()`).
Store	**SQL / Graph DBs**	While vector stores are most common, you can also index data in other ways. For SQL, you might index metadata. For a Graph DB, you would index the relationships *between* entities, which is useful for different kinds of retrieval.